Звіт до лабораторної роботи № 2

Паралельні алгоритми стандартної бібліотеки C++: std::reduce (варіант 13)

Виконала: Поліна Харченко, група К-28

Компілятор	GCC (MSYS2, MinGW-w64), C++20, release (-DNDEBUG)	Паралельні політики	oneTBB
Апаратних потоків	hardware_concurrency = 8	Час одного досліду	медіана з 15 запусків після прогріву

1. Мета і постановка

Дослідити швидкодію std::reduce на послідовностях різної довжини: (1) без політики виконання; (2) з політиками seq, unseq, par, par_unseq; (3) у власному паралельному алгоритмі, що ділить вибірку на K частин. Порівняти збірки без оптимізації (**-Oo**) та з максимальною (**-O3**).

2. Дані та методика

Дані. Чотири послідовності цілих чисел з [1; 100000] довжиною 10^4, 10^5, 10^6, 10^7, згенеровані засобами <random> (std::mt19937, uniform_int_distribution) і збережені у *data/numbers_N.txt* (рядок 1 — N, рядок 2 — числа через пробіл). Обидві збірки читають ті самі файли.

Операції. std::reduce приймає не предикат, а бінарну операцію, тому досліджено дві операції різної складності, обидві асоціативні й комутативні: **add** — сума в long long (одне додавання на елемент) та **mul_mod** — добуток за модулем 10^9+7 (множення й остача, на -O3 приблизно в 13 разів дорожча).

Вимірювання. Міряється лише reduce над масивом, уже завантаженим у пам'ять; читання файлу міряється окремо (для 10^7 чисел ≈283 мс) і в результати не входить. Кожен результат звіряється з reduce без політики: в усіх режимах і для всіх K відповіді збіглися.

3. Бібліотечний std::reduce

режим	N = 10^4		N = 10^5		N = 10^6		N = 10^7	
	add	mul_mod	add	mul_mod	add	mul_mod	add	mul_mod
no_policy	0,002	0,022	0,015	0,191	0,084	1,07	0,801	10,4
seq	0,002	0,022	0,014	0,169	0,084	1,04	0,803	10,4
unseq	0,002	0,015	0,015	0,116	0,085	0,706	0,835	7,11
par	0,014	0,018	0,017	0,041	0,033	0,208	0,650	1,96
par_unseq	0,014	0,019	0,013	0,037	0,028	0,160	0,705	1,47

Таблиця 1. Час std::reduce у збірці -O3, мс.

Для **mul_mod** на 10^7 елементах par скорочує час з 10,4 до 1,96 мс, а par_unseq — до 1,47 мс (у 7,0 раза); однопотоковий unseq за рахунок SIMD дає прискорення в 1,46 раза. Для **add** виграш значно менший: на 10^6 (4 МБ, вміщується в кеш) par прискорює в 2,5 раза, на 10^7 (40 МБ) — лише в 1,23 раза. Тут одне додавання на 4 байти даних, тож швидкість визначає читання з оперативної пам'яті (≈50 ГБ/с уже в одному потоці). На 10^4 паралельні політики програють (0,014 проти 0,002 мс): розподіл задач у TBB дорожчий за саму роботу. no_policy і seq практично однакові, бо обидва виконуються в одному потоці.

4. Власний паралельний алгоритм SplitReducer

Масив з 10^7 елементів ділиться на K частин довжиною N / K (остача дістається останній частині). Кожна частина зводиться послідовним std::reduce в окремому std::thread і записується у власну комірку масиву partial; після join усіх потоків partial зводиться ще одним std::reduce.

K	1	2	3	4	5	6	7	8	10	12	16	24	32	64	128	256	512	1024
add, -O3	0,858	**0,712**	0,719	0,730	0,819	0,789	0,779	0,773	0,795	0,801	0,819	0,829	0,848	0,928	1,35	2,37	5,50	12,1
mul_mod, -O3	10,5	5,47	3,79	2,91	3,40	2,89	2,53	2,33	2,31	2,60	**2,23**	2,51	2,47	2,42	2,78	3,77	5,70	11,5
add, -Oo	16,3	8,44	5,86	4,48	5,67	4,79	4,18	3,75	3,78	3,66	3,79	**3,62**	3,74	3,79	3,97	4,56	6,33	12,3
mul_mod, -Oo	24,4	12,6	8,73	6,65	8,93	7,53	6,59	5,90	6,16	5,86	5,73	5,82	**5,70**	5,78	5,99	6,75	8,23	13,3

Таблиця 2. Час SplitReducer, мс; жирним виділено мінімум у рядку.

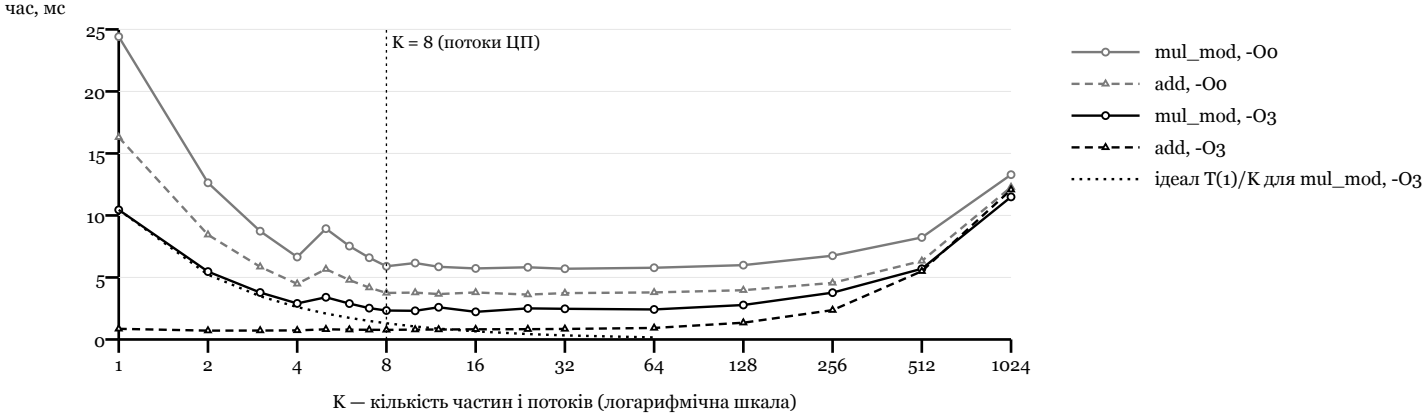


Рисунок 1. Залежність часу від K. Пунктир — ідеальне прискорення T(1)/K.

5. Відповіді на запитання

1. **На якому K досягається найкраща швидкість?** Формальний мінімум: K = 16 для mul_mod, -O3 (2,23 мс); K = 32 для mul_mod, -O0; K = 24 для add, -O0; K = 2 для add, -O3. Проте вже з K = 8 (mul_mod, -O3) час відрізняється від найкращого менш ніж на 5 %, а на проміжку K = 8…64 відмінності на рівні похибки, і найкраще K від запуску до запуску змінюється.
2. **Як це співвідноситься з кількістю потоків процесора?** Достатньо K ≈ 8, тобто 1,00 × hardware_concurrency (8). Менше частин залишає частину ядер без роботи, а більше не додає обчислювальних ресурсів, лише накладні витрати на потоки. Максимальне прискорення — у 4,7 раза, а не в 8: логічні потоки нерівноцінні (два можуть ділити одне фізичне ядро) і користуються спільною пам'яттю. Для add, -O3 найкраще лише 1,21 раза при K = 2: операцію обмежує пропускна здатність пам'яті.
3. **За яким законом зростає час за збільшення K?** До K = 4 час спадає майже як T(1)/K (mul_mod, -O3: 10,5, 5,47 і 2,91 мс для K = 1, 2, 4), далі виходить на плато, а з K ≈ 128 зростає лінійно: кожен додатковий потік коштує ≈11 мкс на створення та join (при K = 512 і 1024: 5,70 і 11,5 мс). Отже, $T(K) \approx T(1) / \min(K, P) + c \cdot K$, де P — кількість ефективно задіяних ядер, c — ціна одного потоку. Бібліотечний par_unseq (1,47 мс) швидший за найкращий SplitReducer (2,23 мс): TBB використовує готовий пул потоків, динамічно перерозподіляє роботу і векторизує цикл.

6. Вплив рівня оптимізації

	add, -O0	add, -O3	-O3 швидше	mul_mod, -O0	mul_mod, -O3	-O3 швидше
no_policy	16,3	0,801	у 20,3 раза	24,2	10,4	у 2,3 раза
seq	24,2	0,803	у 30,2 раза	34,2	10,4	у 3,3 раза
unseq	39,9	0,835	у 47,8 раза	41,0	7,11	у 5,8 раза
par	5,21	0,650	у 8,0 раза	7,39	1,96	у 3,8 раза
par_unseq	8,33	0,705	у 11,8 раза	9,04	1,47	у 6,1 раза
SplitReducer, найкраще K	3,62	0,712	у 5,1 раза	5,70	2,23	у 2,6 раза

Таблиця 3. Час для $N = 10^7$, мс.

-O3 прискорює add у 20 разів, а mul_mod — лише в 2,3 раза: ділення з остачею дороге за будь-якої оптимізації. На -O0 seq і unseq навіть повільніші за no_policy, бо без вбудовування кожен шар обгорток бібліотеки виконується окремим викликом. Без оптимізації add перестає впиратися в пам'ять, і SplitReducer прискорює її в 4,5 раза проти 1,21 на -O3. Тому режими виконання порівнювались лише в межах одного рівня оптимізації.

7. Висновки

- Паралельний reduce виправданий, коли операція обчислювально важка і даних багато: для mul_mod виграш досягає 7,0 раза. Для простої суми великого масиву межею є пропускна здатність пам'яті, а на малих масивах — накладні витрати на потоки.
- Для власного алгоритму достатньо K ≈ 1,00 × кількість апаратних потоків (8); більші K до ≈64 не дають виграшу, а далі лінійно погіршують час через створення потоків.
- Рівень оптимізації змінює час на порядок і може змінити саму природу задачі (з обчислювальної на обмежену пам'яттю), тому впливає на висновки не менше, ніж паралелізм.